

Code Explanation 2ndLidarAttempt

Introduction

There are two versions of the code:

- **original** – The original distance-following implementation, which was later modified into an anti-collision system.
- **2ndLidarAttempt** – A second implementation that adds LiDAR-based object tracking. This version is not fully functional due to suspected issues with the LiDAR sensor.

This document explains the structure and functionality of the **2ndLidarAttempt** code to help future project groups understand how it works and continue its development.

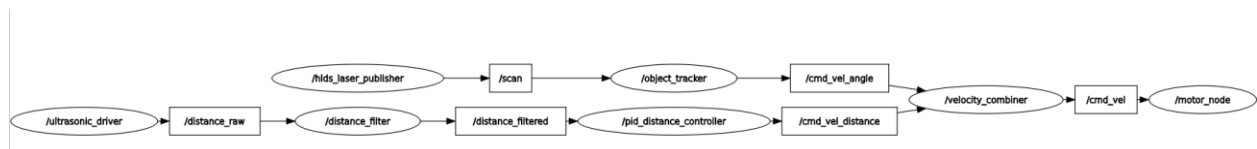
Note: This document is intended as technical documentation and is **not** a final project report.

(Made by Sybren)

Overview

This software consists of several ROS 2 nodes that together implement a distance-following robot. An additional LiDAR-based object tracking node was also developed to allow the robot to follow a moving object laterally (left and right). At the time of writing, the LiDAR node has not been fully tested due to hardware issues with the sensor, which will be discussed later in this report.

RQT Graph



/hids_laser_publisher is from a library that prints the values of the lidar in a array in `/scan`, since this is a preexisting node we I won't explain this further.

When only the nodes following the `ultrasonic_driver` are executed, the robot operates as a distance follower. The data flow between the nodes is shown in the RQT graph.

The Creators from these nodes are:

ultrasonic_driver

distance_filterd

pid_distance_controller

velocity_combiner (inspiration taken from earlier version Mohamed)

object_tracer (inspiration taken from earlier version Mohamed)

made by Sybren

motor_node

by Mohamed

and

hids_laser_publisher from a preexisting library

ultrasonic_driver

This node interfaces directly with the ultrasonic sensor. It periodically sends a 10 μ s trigger pulse and measures the time until the echo is received. Using the measured round-trip travel time and the speed of sound, it calculates the distance to the object in front of the robot. The measured distance is published on the `/distance_raw` topic.

```
import rclpy
```

```
from rclpy.node import Node
from std_msgs.msg import Float32
import time
import lgpio
import sys
```

```

class UltrasonicDriverNode(Node):
    def __init__(self):
        super().__init__('ultrasonic_driver')
        self.publisher_ = self.create_publisher(Float32, 'distance_raw', 10)
        self.timer = self.create_timer(0.04, self.measure_distance) # 25Hz

        # Open gpiochip0 to share access cleanly with your motor node
        try:
            self.chip = lgpio.gpiochip_open(0)
        except Exception as e:
            self.get_logger().error(f"Could not open gpiochip0: {e}")
            sys.exit(1)

        # GPIO Pins Setup (Your chosen pins)
        self.TRIG = 19
        self.ECHO = 26

        lgpio.gpio_claim_output(self.chip, self.TRIG)
        lgpio.gpio_claim_input(self.chip, self.ECHO)

        # Ensure trigger starts low
        lgpio.gpio_write(self.chip, self.TRIG, 0)
        time.sleep(0.1)
        self.get_logger().info("Ultrasonic LGPIO Driver Started.")

    def measure_distance(self):
        # Trigger pulse (10 microseconds)
        lgpio.gpio_write(self.chip, self.TRIG, 1)
        time.sleep(0.00001)
        lgpio.gpio_write(self.chip, self.TRIG, 0)

        start_time = time.time()
        timeout = start_time + 0.04 # ~40ms max round trip safeguard

        # Wait for echo pulse start
        while lgpio.gpio_read(self.chip, self.ECHO) == 0:
            start_time = time.time()
            if start_time > timeout:
                self.get_logger().error("Sensor Timeout: Echo start not
detected")
                return

        # Wait for echo pulse end

```

```

        stop_time = time.time()
        while lgpio.gpio_read(self.chip, self.ECHO) == 1:
            stop_time = time.time()
            if stop_time > timeout:
                break # Sensor range exceeded

        duration = stop_time - start_time
        # Speed of sound is 34300 cm/s. Divide by 2 for round-trip path.
        distance = (duration * 34300.0) / 2.0

        msg = Float32()
        msg.data = float(distance)
        self.publisher_.publish(msg)

def main(args=None):
    rclpy.init(args=args)
    node = UltrasonicDriverNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

distance_filter

Raw ultrasonic measurements often contain noise and occasional invalid readings. This node first validates each measurement by checking whether it falls within the operating range specified in the sensor datasheet (3–380 cm). Invalid measurements are discarded.

The remaining values are processed using a median filter with a depth of three samples, which effectively removes spikes while preserving the actual distance. The filtered distance is then published on the `/distance_filtered` topic.

```
import rclpy

from rclpy.node import Node
from std_msgs.msg import Float32

class DistanceFilterNode(Node):
    def __init__(self):
        super().__init__('distance_filter')
        # Listen to 'distance_raw'
        self.ym_subscription = self.create_subscription(Float32, '/distance_raw',
self.filter_callback, 10)
        # Publish on 'distance_filtered'
        self.ym_publisher = self.create_publisher(Float32, '/distance_filtered',
10)

        # Filter State
        self.ym_data_buffer = [0.0, 0.0, 0.0]
        self.ym_min_val = 3.0
        self.ym_max_val = 380.0

    def filter_callback(self, msg):
        ym_raw_val = msg.data

        # 1. Range Validation
        if ym_raw_val < self.ym_min_val or ym_raw_val > self.ym_max_val:
            out_msg = Float32()
            out_msg.data = 0.0
            self.ym_publisher.publish(out_msg)
            self.get_logger().warn(f'Out of range: {ym_raw_val:.2f} cm')
            return

        # 2. Median Filter (Depth 3)
        self.ym_data_buffer.pop(0)
        self.ym_data_buffer.append(ym_raw_val)

        sorted_data = sorted(self.ym_data_buffer)
        ym_median_val = sorted_data[1]
```

```
        # 3. Publish
        out_msg = Float32()
        out_msg.data = ym_median_val
        self.ym_publisher.publish(out_msg)
        self.get_logger().info(f'Filtered Median: {ym_median_val:.2f} cm')

def main(args=None):
    rclpy.init(args=args)
    ym_node = DistanceFilterNode()
    try:
        rclpy.spin(ym_node)
    except KeyboardInterrupt:
        pass
    finally:
        ym_node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

pid_distance_controller

This node receives the filtered distance and compares it with a desired setpoint of 20 cm. A PID controller calculates the required forward or backward velocity needed to maintain this distance. The controller includes proportional, integral, and derivative terms, as well as an anti-windup mechanism to prevent excessive accumulation of the integral term. The calculated linear velocity is published as a Twist message on the `/cmd_vel_distance` topic.

```
import rclpy

from rclpy.node import Node

from std_msgs.msg import Float32
from geometry_msgs.msg import Twist

class PIDDistanceController(Node):

    def __init__(self):
        super().__init__('pid_distance_controller')

        self.cmd_pub = self.create_publisher(
            Twist,
            '/cmd_vel_distance',
            10
        )

        self.subscription = self.create_subscription(
            Float32,
            '/distance_filtered',
            self.value_callback,
            10
        )

        # Desired distance (cm)
        self.setpoint = 20.0

        # PID gains
        self.kp = 0.22    # Proportional response
```

```

self.ki = 0.001 # Integral response
self.kd = 0.02  # Derivative damping

self.integral = 0.0
self.prev_error = 0.0

# Anti-Windup Limit: Prevents the integral term from growing infinitely
# and overriding the Proportional/Derivative reverse commands.
self.max_integral = 5.0

self.prev_distance = None

# Use ROS 2 System Clock instead of Python wall time
self.prev_time = self.get_clock().now()

self.get_logger().info("PID Distance Controller Started with Anti-Windup
Fix.")

def value_callback(self, msg):
    distance = msg.data

    # Safeguard against invalid or negative sensor readings
    if distance <= 0.0:
        return

    current_time = self.get_clock().now()

    # Calculate dt in seconds from ROS 2 Duration
    dt = (current_time - self.prev_time).nanoseconds / 1e9

    # Prevent division by zero or processing if callbacks arrive too
    instantly
    if dt <= 0.001:
        return

    # -----
    # PID CONTROL
    # -----
    error = self.setpoint - distance

    # Accumulate integral and clamp it immediately (Anti-Windup)
    self.integral += error * dt

```

```

        self.integral = max(min(self.integral, self.max_integral), -
self.max_integral)

    # Calculate derivative
    derivative = (error - self.prev_error) / dt

    # Raw PID Control Output
    output = (
        self.kp * error +
        self.ki * self.integral +
        self.kd * derivative
    )

    # -----
    # OBJECT SPEED ESTIMATE
    # -----
    object_speed = 0.0
    if self.prev_distance is not None:
        object_speed = (distance - self.prev_distance) / dt

    # -----
    # LIMIT SPEED & INVERSION MANAGEMENT
    # -----
    max_speed = 1.0

    # Flip the sign (-output) to match your motor/hardware orientation
requirements
    cmd_speed = max(min(-output, max_speed), -max_speed)

    # Publish the movement command
    twist = Twist()
    twist.linear.x = cmd_speed
    self.cmd_pub.publish(twist)

    # Logging output for live debugging and monitoring
    self.get_logger().info(
        f"D={distance:.2f}cm | Err={error:.2f} | Vel={cmd_speed:.2f} |
ObjSpeed={object_speed:.2f} cm/s"
    )

    # Save states for the next incoming callback
    self.prev_error = error
    self.prev_distance = distance

```

```
        self.prev_time = current_time

def main(args=None):
    rclpy.init(args=args)
    node = PIDDistanceController()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        # Stop the robot immediately on shutdown
        stop_twist = Twist()
        node.cmd_pub.publish(stop_twist)

        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

velocity_combiner

The velocity combiner subscribes to both the linear velocity generated by the PID distance controller and the angular velocity generated by the LiDAR object tracker. It combines these two independent commands into a single Twist message containing both linear and angular velocities. This combined command is published on the `/cmd_vel` topic.

```
import rclpy

from rclpy.node import Node
from geometry_msgs.msg import Twist

class VelocityCombiner(Node):
    def __init__(self):
        super().__init__('velocity_combiner')

        # Latest received commands
        self.linear_cmd = 0.0
        self.angular_cmd = 0.0

        # Subscriptions
        self.dist_sub = self.create_subscription(
            Twist, '/cmd_vel_distance', self.dist_callback, 10)
        self.angle_sub = self.create_subscription(
            Twist, '/cmd_vel_angle', self.angle_callback, 10)

        # Publisher to the motor node
        self.twist_pub = self.create_publisher(Twist, '/cmd_vel', 10)

        # Timer to publish at a steady rate (e.g., 20Hz)
        self.timer = self.create_timer(0.05, self.publish_combined_vel)

    def dist_callback(self, msg):
        # Grab only the linear movement
        self.linear_cmd = msg.linear.x

    def angle_callback(self, msg):
        # Grab only the angular movement
        self.angular_cmd = msg.angular.z
```

```
def publish_combined_vel(self):
    combined_msg = Twist()
    combined_msg.linear.x = self.linear_cmd
    combined_msg.angular.z = self.angular_cmd

    self.twist_pub.publish(combined_msg)

def main(args=None):
    rclpy.init(args=args)
    node = VelocityCombiner()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

motor_node

The motor node receives the combined velocity command and converts the linear and angular velocities into individual left and right wheel speeds using differential drive kinematics. It generates the required PWM signals and direction outputs through the GPIO pins to drive the motor controller, allowing the robot to move forward, backward, or turn while maintaining the desired distance from the target.

```
import rclpy
```

```
from rclpy.node import Node
from geometry_msgs.msg import Twist
import lgpio
import sys

class MotorNode(Node):
    def __init__(self):
        super().__init__('motor_node')

        # Open gpiochip0 (Confirmed by your 'ls /dev/gpiochip*' command)
        try:
            self.chip = lgpio.gpiochip_open(0)
        except Exception as e:
            self.get_logger().error(f"Could not open gpiochip0: {e}")
            sys.exit(1)

        # Pinout mapping
        # ENA/ENB are PWM. IN1-IN4 are Direction pins.
        self.pins = {
            'L_PWM': 18, 'L_IN1': 17, 'L_IN2': 27,
            'R_PWM': 23, 'R_IN1': 22, 'R_IN2': 24
        }

        # Claim all pins as outputs
        for pin in self.pins.values():
            lgpio.gpio_claim_output(self.chip, pin)

        # Subscriber for velocity commands
        self.subscription = self.create_subscription(
            Twist,
```

```

        '/cmd_vel',
        self.listener_callback,
        10)

    self.get_logger().info("Motor Node Ready. Directions flipped for forward
motion.")

def set_motor(self, pwm_pin, in1, in2, speed):
    # Limit speed between -1.0 and 1.0
    speed = max(min(speed, 1.0), -1.0)
    duty = abs(speed) * 100

    # PWM at 500Hz for better motor startup torque
    lgpio.tx_pwm(self.chip, pwm_pin, 500, duty)

    # DIRECTION LOGIC (Flipped 0/1 to fix the 'moving backward' issue)
    if speed > 0:
        lgpio.gpio_write(self.chip, in1, 0)
        lgpio.gpio_write(self.chip, in2, 1)
    elif speed < 0:
        lgpio.gpio_write(self.chip, in1, 1)
        lgpio.gpio_write(self.chip, in2, 0)
    else:
        lgpio.gpio_write(self.chip, in1, 0)
        lgpio.gpio_write(self.chip, in2, 0)

def listener_callback(self, msg):
    linear = msg.linear.x
    angular = msg.angular.z

    # FIX: Adjusted the directional signs so "linear" drives both wheels
forward
    # and "angular" smoothly rotates them in opposite directions.
    left_speed = linear + angular
    right_speed = linear - angular

    self.set_motor(self.pins['L_PWM'], self.pins['L_IN1'],
self.pins['L_IN2'], left_speed)
    self.set_motor(self.pins['R_PWM'], self.pins['R_IN1'],
self.pins['R_IN2'], right_speed)

def main(args=None):
    rclpy.init(args=args)

```

```
node = MotorNode()
try:
    rclpy.spin(node)
except KeyboardInterrupt:
    # Emergency Stop on Ctrl+C
    node.set_motor(node.pins['L_PWM'], node.pins['L_IN1'],
node.pins['L_IN2'], 0)
    node.set_motor(node.pins['R_PWM'], node.pins['R_IN1'],
node.pins['R_IN2'], 0)
    node.get_logger().info("Motors Stopped.")

node.destroy_node()
rclpy.shutdown()

if __name__ == '__main__':
    main()
```

object_tracker (LiDAR node)

This node is intended to process data from the LiDAR sensor to detect and track the closest object within a 30° field of view in front of the robot. The angle of the detected object is compared with the desired forward direction, and a proportional controller generates an angular velocity command that steers the robot towards the object. This command is published on the `/cmd_vel_angle` topic. Due to hardware issues with the LiDAR, this node has not yet been fully tested or verified.

```
import rclpy

from rclpy.node import Node
from sensor_msgs.msg import LaserScan
from geometry_msgs.msg import Twist
from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy
import numpy as np
import sys

class ObjectTracker(Node):
    def __init__(self):
        super().__init__('object_tracker')

        # Best Effort QoS voor de LiDAR
        ym_qos_profile = QoSProfile(
            reliability=ReliabilityPolicy.BEST_EFFORT,
            history=HistoryPolicy.KEEP_LAST,
            depth=10
        )

        self.ym_subscription = self.create_subscription(
            LaserScan,
            '/scan',
            self.ym_listener_callback,
            ym_qos_profile)

        self.ym_publisher = self.create_publisher(Twist, '/cmd_vel_angle', 10)

        self.ym_target_angle = 0.0
        self.ym_Kp = 0.5
        self.ym_max_steer = 0.5
        self.ym_fov_rad = np.radians(30.0)
```

```

        # NIEUW: Onafhankelijke timer die ALTIJD moet draaien
        self.ym_heartbeat_timer = self.create_timer(1.0,
self.ym_heartbeat_callback)

        print("----> TERMINAL: Object Tracker is succesvol geïntialiseerd!",
file=sys.stderr, flush=True)

    def ym_heartbeat_callback(self):
        print("----> TERMINAL HEARTBEAT: De ROS 2 Event Loop (spin) draait
correct!", file=sys.stderr, flush=True)

    def ym_listener_callback(self, msg):
        ym_ranges = np.array(msg.ranges)

        # Maak de array van hoeken aan
        ym_angles = msg.angle_min + (np.arange(len(ym_ranges)) *
msg.angle_increment)

        # SOFTWAREMATIGE INVERSIE: Voeg pi toe om de 0 graden naar de overkant te
flippen!
        ym_angles = ym_angles + np.pi

        # Normaliseer de hoeken weer netjes tussen -pi en pi
        ym_angles = np.arctan2(np.sin(ym_angles), np.cos(ym_angles))

        # ---- Vanaf hier blijft de rest van je cone-filtering exact hetzelfde --
--
        ym_valid_indices = np.where(
            (ym_ranges > msg.range_min) &
            (ym_ranges < msg.range_max) &
            (ym_angles >= -self.ym_fov_rad) &
            (ym_angles <= self.ym_fov_rad)
        )[0]

        if len(ym_valid_indices) == 0:
            print("----> DEBUG: Niets gezien in de 30 graden cone vooruit.",
file=sys.stderr, flush=True)
            return

        ym_closest_index =
ym_valid_indices[np.argmin(ym_ranges[ym_valid_indices])]

```

```

        ym_object_angle = ym_angles[ym_closest_index]

        ym_error = ym_object_angle - self.ym_target_angle
        ym_steering_cmd = self.ym_Kp * ym_error
        ym_steering_cmd = np.clip(ym_steering_cmd, -self.ym_max_steer,
self.ym_max_steer)

        ym_cmd_msg = Twist()
        ym_cmd_msg.angular.z = float(ym_steering_cmd)
        self.ym_publisher.publish(ym_cmd_msg)

        print(f"----> TRACKING: Angle: {np.degrees(ym_object_angle):.1f}° |
Steer: {ym_steering_cmd:.2f}", file=sys.stderr, flush=True)
def main(args=None):
    rclpy.init(args=args)
    ym_node = ObjectTracker()
    try:
        rclpy.spin(ym_node)
    except KeyboardInterrupt:
        pass
    finally:
        ym_node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

LiDAR Issues

The original goal of the LiDAR node was to detect the closest object in front of the robot and generate an angular velocity command, allowing the robot to follow a moving object while also tracking its left and right movement. However, during testing the LiDAR sensor did not produce the expected measurements, preventing the object tracking algorithm from functioning correctly.

To investigate the issue, the original callback function was temporarily replaced with a diagnostic version. Instead of calculating a steering command, this function divided the 360° scan into eight sectors of 45° each and calculated the average distance and the number of valid laser returns within every sector. Infinite (`inf`) values were temporarily replaced with zero to simplify the calculations and identify where valid measurements were being received.

The diagnostic output consistently showed that only the sectors between approximately 225° and 360° contained valid measurements, while the entire front half of the scan (0°–225°) returned no valid data. A typical output is shown below.

```
Index 000°–045° : Average = 0.00 m (0/45 valid rays)
Index 045°–090° : Average = 0.00 m (0/45 valid rays)
Index 090°–135° : Average = 0.00 m (0/45 valid rays)
Index 135°–180° : Average = 0.00 m (0/45 valid rays)
Index 180°–225° : Average = 0.00 m (0/45 valid rays)
Index 225°–270° : Average = 3.23 m (17/45 valid rays)
Index 270°–315° : Average = 1.29 m (45/45 valid rays)
Index 315°–360° : Average = 1.54 m (45/45 valid rays)
```

During testing, moving the robot or the surrounding objects produced little to no change in the sectors that should have been observing the environment. Instead, the valid measurements remained concentrated in the same region of the scan, indicating that the sensor was not providing a complete 360° scan.

To verify whether the problem was caused by the software or by the hardware, the LiDAR sensor was physically rotated by 90° and the same diagnostic test was repeated. The results showed that the valid measurement region remained the same (225–360 deg).

Based on these observations, it was concluded that the issue originated from the LiDAR hardware rather than the ROS 2 software. Since only part of the scan contained valid measurements, the object tracking algorithm could not reliably determine the angle of the closest object. As a result,

the `object_tracker` node could not be fully tested or validated, and only the ultrasonic distance-following functionality was verified during this project. I currently have no idea why Mohamed's version worked since both used the same library and QosProfile.

To reproduce this test, replace the original `ym_listener_callback()` function in the `LiDAR` node with the following diagnostic callback:

```
def ym_listener_callback(self, msg):

    ym_ranges = np.array(msg.ranges)

    # Replace infinite values with 0.0 for easier calculations
    ym_ranges_clean = np.where(ym_ranges == float('inf'), 0.0, ym_ranges)

    print("\n===== SEARCHING FOR THE WALL =====", file=sys.stderr,
flush=True)

    # Divide the scan into 45° sectors and analyse each sector
    for ym_i in range(0, 360, 45):
        ym_slice = ym_ranges_clean[ym_i:ym_i+45]

        # Calculate the average of all valid measurements
        ym_valid_elements = ym_slice[ym_slice > 0.0]
        ym_avg = np.mean(ym_valid_elements) if len(ym_valid_elements) > 0
    else 0.0
        ym_hits = len(ym_valid_elements)

        print(
            f"Index {ym_i:03d}° to {(ym_i+45):03d}°: "
            f"Average = {ym_avg:.2f} m ({ym_hits}/45 valid rays)",
            file=sys.stderr,
            flush=True
        )

    print("=====\n", file=sys.stderr,
flush=True)
```

Running the program

For ease of use the next piece of code was used to run the program without having to reload the workspace. Running this code will use all latest versions of .py files

```
#!/bin/bash

# Clear trap to handle emergency shutdown sequence cleanly
trap 'echo -e "\nExecuting Emergency System Shutdown Sequence...";
ym_kill_all_nodes' SIGINT SIGTERM EXIT

ym_kill_all_nodes() {
    # Block recursive traps during cleanup
    trap - SIGINT SIGTERM EXIT

    echo "Stopping all tracked background nodes..."
    for ym_pid in "${ym_pids[@]}; do
        if kill -0 "$ym_pid" 2>/dev/null; then
            kill -15 "$ym_pid" 2>/dev/null
        fi
    done

    # Backup cleanup for safety
    pkill -f motor_controller 2>/dev/null
    pkill -f hlds_laser_publisher 2>/dev/null
    echo "All nodes completely cleaned from background memory."
    exit 0
}

cd ~/ros2_ws

# 1. Build first. If C++ code changed, this compiles it.
colcon build --symlink-install
source /opt/ros/humble/setup.bash
```

```

source install/local_setup.bash

echo "Launching Core Autonomous Navigation Nodes..."
ym_pids=()

echo "Nodes worden opgestart..."

# -----
# 2. Start alle nodes APART op en sla de PIDs op
# -----

# Hardware Node: LiDAR-driver met expliciete poort en frame-ID (Output
onderdrukt)
ros2 run hls_lfcd_lds_driver hlds_laser_publisher --ros-args -p
port:=/dev/ttyUSB0 -p frame_id:=laser > /dev/null 2>&1 &
ym_pids+=($!)

# Node 1: UltraSonic Driver (Output onderdrukt)
ros2 run motor_controller driverUltra_node > /dev/null 2>&1 &
ym_pids+=($!)

# Node 2: Median Node (Output onderdrukt)
ros2 run motor_controller median_node > /dev/null 2>&1 &
ym_pids+=($!)

# Node 3: Lidar Avoidance (VOLLEDIGE OUTPUT IN DE TERMINAL)
env PYTHONUNBUFFERED=1 ros2 run motor_controller lidar_avoidance &
ym_pids+=($!)

# Node 4: PID Test Node (Output onderdrukt)
ros2 run motor_controller PIDtest_node > /dev/null 2>&1 &
ym_pids+=($!)

# Node 5: Motor Node (Output onderdrukt)
ros2 run motor_controller motor_node > /dev/null 2>&1 &
ym_pids+=($!)

# Node 6: Mixer Node (Output onderdrukt)
ros2 run motor_controller mixer_node > /dev/null 2>&1 &
ym_pids+=($!)

# -----

```

```
echo "Alle nodes draaien correct via /dev/ttyUSB0. Druk op [CTRL+C] om ze  
allemaal veilig te stoppen."
```

```
# Keep script alive until interrupted  
wait
```

Conclusion

The 2ndLidarAttempt project extends the original distance-following implementation by introducing LiDAR-based object tracking. The software architecture separates the different tasks into ROS 2 nodes, making the system modular and easier to understand, test, and modify.

The ultrasonic distance-following system operates as intended and demonstrates how sensor data is processed, filtered, converted into velocity commands, and finally translated into motor control. Although the LiDAR-based tracking functionality could not be fully validated due to suspected hardware issues, the software implementation provides a solid foundation for future development.

This document was written to help future project groups understand the purpose of each node, the communication between them, and the challenges encountered during development. With a functioning LiDAR sensor, the remaining functionality can be tested and further improved.